
Classification in Machine Learning

Theory Paper — Part A

1. Introduction to Classification

Classification is a supervised machine learning task in which an algorithm learns to assign input examples to one of a predefined set of discrete categories, known as classes or labels. The algorithm is trained on a labelled dataset — a collection of examples for which the correct class is already known — and uses that experience to predict the class of new, unseen examples.

1.1 Classification vs. Regression

Although both classification and regression are forms of supervised learning, they differ fundamentally in the nature of their output. Regression predicts a **continuous numeric value** — a number on an infinite scale — whereas classification predicts a **discrete category** from a finite set of possible outputs. The key distinction is not in the algorithm family but in what the model is trying to estimate: a real-valued quantity or a class membership probability.

Consider the following contrasting examples:

- **Classification example:** A bank uses a model to predict whether a loan application should be Approved (1) or Rejected (0), based on the applicant's credit score, income, employment history, and collateral. The output is one of two categories.
- **Regression example:** An estate agent uses a model to predict the selling price of a house in Egyptian pounds, based on its area, number of rooms, location, and age. The output is a continuous number with no fixed upper bound.

2. Classification Algorithms

The following section describes three widely used classification algorithms, comparing their underlying mechanisms, practical applications, and trade-offs.

2.1 Logistic Regression

Despite its name, Logistic Regression is a classification algorithm, not a regression model. It models the probability that a given input belongs to a particular class by fitting a **linear decision boundary** to the feature space and passing the raw linear output through a **sigmoid function**, which maps any real number to the interval (0, 1). The resulting probability is then thresholded — typically at 0.5 — to produce a binary prediction.

The model is trained by minimizing a loss function called binary cross-entropy using gradient descent, iteratively adjusting the weights assigned to each feature until the predicted probabilities align as closely as possible with the true labels.

Real-world use case: Credit scoring — predicting whether a loan applicant will default based on their financial history. The model's coefficients directly indicate which features increase or decrease the probability of default, making the output auditable by regulators.

Advantages: Fast to train, computationally cheap, outputs calibrated probabilities rather than just labels, and produces interpretable coefficients that show the direction and magnitude of each feature's influence.

Limitations: Can only learn linear decision boundaries — it will underperform when the true boundary between classes is curved or involves interactions between features. It also requires feature scaling and is sensitive to outliers in the input data.

2.2 Decision Trees

A Decision Tree is a non-parametric model that learns a hierarchy of **if/else rules** by recursively splitting the training data on the feature that produces the greatest separation between classes at each step. The quality of a split is typically measured using **Gini impurity** or **information gain** (entropy reduction). The process continues until a stopping criterion is met — such as a maximum depth or a minimum number of samples per leaf — at which point each leaf node is assigned the majority class of the examples it contains.

Real-world use case: Medical diagnosis — a Decision Tree can learn rules such as 'if blood pressure > 140 and age > 50 and cholesterol > 200, predict high cardiac risk.' Clinicians can read and validate these rules directly, which is critical in healthcare settings.

Advantages: Produces human-readable rules that require no statistical knowledge to interpret; handles both numeric and categorical features natively; requires no feature scaling; and can model non-linear boundaries.

Limitations: Highly prone to overfitting when grown without constraints — an unconstrained tree can memorize every training example, producing near-perfect training accuracy but poor generalization. The model is also unstable: small changes in the training data can produce a completely different tree structure.

2.3 Random Forest

Random Forest is an **ensemble method** that addresses the overfitting and instability problems of individual Decision Trees. It trains a large number of trees (typically 100 to 1,000) on different random subsets of the training data (a technique called **bagging**) and, at each split within each tree, considers only a random subset of features. The final prediction is made by aggregating the votes of all trees — the class receiving the most votes wins.

This combination of randomness in data sampling and feature selection ensures that individual trees are different from one another. When their predictions are averaged, individual errors tend to cancel out, resulting in a model that is both more accurate and more stable than any single tree.

Real-world use case: Fraud detection in financial transactions — the model can simultaneously evaluate dozens of features (transaction amount, time, location, merchant category, deviation from historical behavior) to flag suspicious activity, without requiring the rules to be explicitly programmed.

Advantages: Achieves high accuracy across a wide range of datasets; very resistant to overfitting; provides a built-in estimate of feature importance; and requires minimal hyperparameter tuning to produce strong baselines.

Limitations: Significantly slower to train and predict than a single Decision Tree, especially with large numbers of estimators; the resulting model is a 'black box' — it is not possible to extract a single interpretable set of rules; and it is memory-intensive.

Algorithm	How it works	Advantages	Limitations
Logistic Regression	Finds a linear boundary; passes output through sigmoid to produce a 0–1 probability	Fast, interpretable coefficients, outputs probabilities	Only linear boundaries; needs feature scaling; poor with interactions
Decision Tree	Splits data on features that best separate classes; builds a tree of if/else rules	Human-readable rules, no scaling needed, handles mixed types	Overfits easily; unstable to small data changes
Random Forest	Builds many trees on random data/feature subsets; votes on final prediction	High accuracy, rarely overfits, built-in feature importance	Slower to train; hard to interpret individual rules

3. Classification Metrics

Evaluating a classification model requires more than a single number. Different metrics highlight different aspects of model behavior, and the appropriate metric depends on the nature of the problem and the cost of different types of error. All of the following metrics are derived from the **confusion matrix**, which tabulates the four possible prediction outcomes.

3.1 Confusion Matrix

A confusion matrix is a 2×2 table (for binary classification) that records the full breakdown of predictions against actual labels:

- **True Positives (TP):** The model correctly predicted the positive class (e.g., correctly approved a creditworthy applicant).
- **True Negatives (TN):** The model correctly predicted the negative class (e.g., correctly rejected a risky applicant).
- **False Positives (FP):** The model incorrectly predicted positive (e.g., approved a risky applicant — a false alarm).
- **False Negatives (FN):** The model incorrectly predicted negative (e.g., rejected a creditworthy applicant — a missed opportunity).

All four scalar metrics below are computed directly from these four values.

3.2 Accuracy

Accuracy is the proportion of all predictions that were correct: $(TP + TN) / \text{Total}$. It is the most intuitive metric and is appropriate when both classes are equally represented in the data and both types of error carry equal cost. However, it becomes severely misleading in imbalanced datasets, as discussed in Section 4.

3.3 Precision

Precision measures the correctness of positive predictions: $TP / (TP + FP)$. It answers the question: 'Of all the applicants the model approved, what fraction actually deserved approval?' High precision means the model rarely generates false alarms — when it says 'yes', it is usually right.

3.4 Recall (Sensitivity)

Recall measures coverage of the positive class: $TP / (TP + FN)$. It answers: 'Of all the applicants who genuinely deserved approval, what fraction did the model actually approve?' High recall means the model rarely misses real positives, even if it also approves some who should be rejected.

3.5 F1-Score

The F1-Score is the harmonic mean of Precision and Recall: $2 \times P \times R / (P + R)$. It is the standard single-number summary when a balance between precision and recall is required. Because it uses the harmonic mean rather than the arithmetic mean, a model that achieves perfect precision but zero recall (or vice versa) will receive an F1 of zero — it cannot be 'gamed' by optimizing one metric at the expense of the other.

Metric	What it measures	Formula	Imbalance sensitivity
Accuracy	Fraction of all correct predictions	$(TP+TN) / \text{Total}$	High — misleading when classes are skewed
Precision	Of predicted positives, how many are real	$TP / (TP+FP)$	Moderate — focuses on false alarms
Recall	Of real positives, how many were found	$TP / (TP+FN)$	Low — focuses on missed cases
F1-Score	Balance of precision and recall	$2 \times P \times R / (P+R)$	Low — penalizes both types of error
Confusion Matrix	Full breakdown of TP, TN, FP, FN	N/A (2x2 table)	None — shows all counts directly

4. Class Imbalance

4.1 Why Accuracy is Misleading

Class imbalance occurs when one class is far more frequent than another — for example, in a dataset where 95% of loan applications are rejected and only 5% are approved. In this scenario, a trivial model that **always predicts the majority class** (Rejected) would achieve 95% accuracy without learning anything at all. Accuracy treats every prediction as equally important regardless of class, so it rewards the model simply for repeating the dominant label.

The confusion matrix immediately exposes this: the 'accurate' trivial model would have zero True Positives and zero recall for the minority class — it never identifies a single positive example. Metrics such as Precision, Recall, and F1-Score are sensitive to this failure because they specifically measure performance on the positive (minority) class.

4.2 Precision vs. Recall in Loan Approval

The choice between prioritizing Precision or Recall depends on the cost of each type of error in the specific business context:

Prioritize Precision when False Positives are costly — that is, when approving a loan to someone who will default is more damaging than rejecting someone who would have repaid. A bank operating in a high-risk environment with limited capital reserves would prefer a conservative model with high precision: it approves fewer loans overall but the ones it approves are very likely to be repaid. Rejecting a genuinely creditworthy applicant (False Negative) is a lost revenue opportunity but does not directly cause a financial loss.

Prioritize Recall when False Negatives are costly — that is, when missing a creditworthy applicant is more damaging than occasionally approving a bad one. A bank entering a new market trying to grow its customer base would prefer high recall: it wants to capture as many qualified applicants as possible, even if it occasionally approves someone who later defaults. The revenue from correctly approved loans outweighs the cost of the occasional bad debt.

In practice, the F1-Score is often used for loan models because it forces a balance between the two: a model cannot inflate its score by simply approving or rejecting everyone.

5. Real-World Case Study: Credit Risk Classification at LendingClub

LendingClub, one of the largest peer-to-peer lending platforms in the United States, has been the subject of numerous published academic and industry classification studies using its publicly released loan data. The following summarizes a representative study by Malekipirbazari and Aksakalli (2015), published in *Expert Systems with Applications*, which used machine learning to predict borrower default on the LendingClub platform.

5.1 Goal

The objective was to build a classification model that could predict, at the time of loan issuance, whether a borrower would fully repay their loan or default. The practical motivation was to give individual investors a tool to select loans with acceptable risk profiles, since LendingClub allowed retail investors to fund loans in small increments.

5.2 Data

The dataset consisted of over 9,500 loans issued between 2007 and 2013, with features including borrower credit grade, annual income, debt-to-income ratio, employment length, loan amount, interest rate, and loan purpose. The target variable was binary: Fully Paid (0) or Defaulted/Charged Off (1). The dataset was class-imbalanced, with defaulted loans representing approximately 20% of records.

5.3 Classifiers Applied

The study evaluated four classification algorithms: **Random Forest**, **Support Vector Machines (SVM)**, Logistic Regression, and k-Nearest Neighbor's (KNN). Random Forest and SVM were found to outperform the other methods. Notably, Random Forest also matched closely with LendingClub's own internal credit grading system in terms of the loans it selected, providing external validation of the approach.

5.4 Key Results

The Random Forest model achieved the highest overall performance, with an AUC (Area Under the ROC Curve) of approximately 0.67 — meaningfully above the 0.5 baseline of random guessing. More importantly, simulated investment portfolios constructed using the model's predictions produced significantly higher returns than portfolios built using LendingClub's own credit grades, suggesting that the model captured risk signals not fully reflected in the official ratings.

The study concluded that ensemble methods such as Random Forest are particularly suited to credit risk problems because they handle the mixed feature types, non-linear interactions, and class imbalance that characterize real lending data — without requiring the explicit feature engineering demanded by linear models.

References

- Malekipirbazari, M. and Aksakalli, V. (2015). Risk assessment in social lending via random forests. *Expert Systems with Applications*, 42(10), pp. 4621–4631. Available at: <https://doi.org/10.1016/j.eswa.2015.02.001>
- Scikit-learn Developers (2024). *sklearn.linear_model.LogisticRegression*. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- Scikit-learn Developers (2024). *sklearn.tree.DecisionTreeClassifier*. Available at: <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
- Scikit-learn Developers (2024). *sklearn.ensemble.RandomForestClassifier*. Available at: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- Brownlee, J. (2020). *Imbalanced Classification with Python*. Machine Learning Mastery. Available at: <https://machinelearningmastery.com/imbalanced-classification-with-python/>